

Kenalan sama AutoGrade

Dokumen ini buat orang baru — manusia atau AI — yang belum pernah nyentuh repo ini. Bacanya sekali duduk, sekitar 20 menit, dan habis itu kamu ngerti sistemnya ngapain, kodenya ditaruh di mana, dan mana yang jangan diutak-atik.

Kalau kamu AI agent: ini titik masuk. `CLAUDE.md` itu peta aturan yang lebih padat, `docs/overview.md` isinya detail dalam. Dokumen ini yang menyambungkan semuanya.

1. Ini sistem apa, sih

AutoGrade itu **mata** pabrik kelapa sawit.

Truk masuk bawa buah sawit (namanya **janjang** atau tandan). Buah itu dituang ke conveyor, lewat di bawah kamera, dan kamera plus AI menilai satu per satu: buah ini layak diterima (**ACC**) atau ditolak (**REJ**). Hasilnya dipakai buat nentuin berapa yang dibayar ke pemasok.

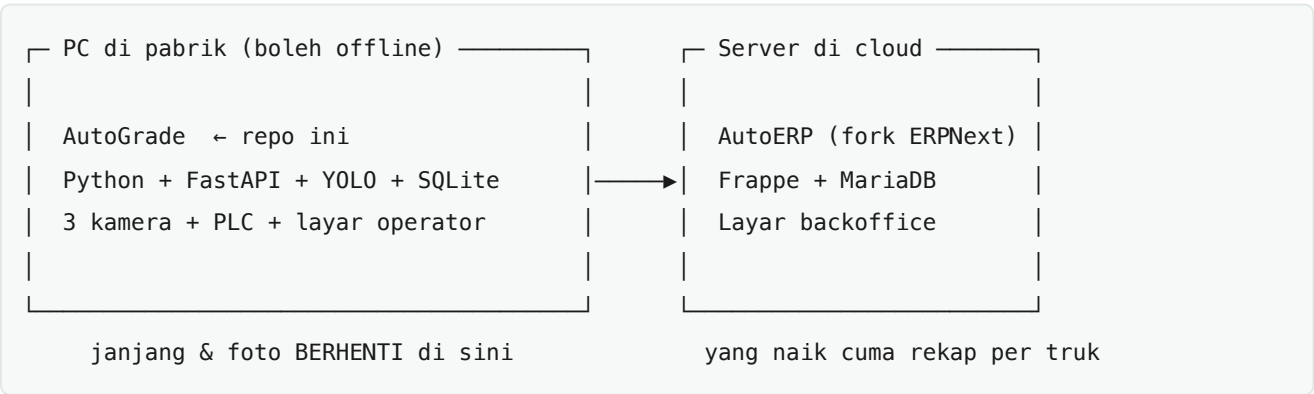
Dulu penilaian ini dikerjain manusia dengan mata telanjang sambil berdiri di panas. Hasilnya beda-beda tiap orang, tiap jam, tiap capek. AutoGrade bikin penilaiannya konsisten dan tercatat — lengkap dengan fotonya, jadi kalau pemasok protes ada buktinya.

Tiga hal yang AutoGrade kerjakan:

- 1. Lihat tiap janjang lewat 3 kamera (3 conveyor, kita sebut **line**), nilai ACC/REJ.
- 2. Kasih layar buat operator: truk mana lagi di line mana, berapa janjang masuk, berat timbangan.
- 3. Kirim **rekap per truk** ke ERP (AutoERP), yang ngurus pembukuan dan pembayaran.

Yang AutoGrade TIDAK kerjakan: harga, potongan, pembayaran, invoice. Itu semua punya ERP. AutoGrade cuma sensor pintar plus layar operator.

2. Dua kotak, jangan ketuker



Kenapa dipisah begitu:

- **Internet di pabrik suka putus.** Kalau layar operator butuh internet, pabrik berhenti waktu internetnya mati. Jadi semua yang operator butuhkan hidup di PC pabrik.
- **ERP itu buku besar, bukan tempat nyimpen event.** Ngirim 4.000 janjang per hari ke ERP bikin dia sesak tanpa guna. Yang ERP butuh cuma: truk ini bawa sekian kilo, sekian persen ditolak. Foto dan detail per janjang tetap di pabrik sebagai bukti.

Arah komunikasinya satu arah: pabrik → ERP. ERP nggak pernah manggil ke pabrik, karena PC pabrik nggak punya pintu masuk dari internet sama sekali.

3. Empat container dari satu image

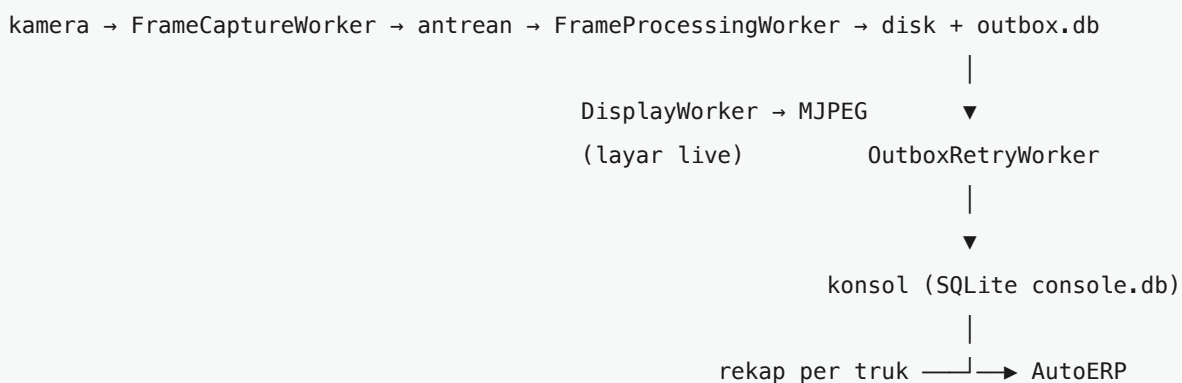
Satu image Docker, dijalankan empat kali dengan peran beda:

Container	Port	Isinya	Kalau mati?
ripe_line_1	8001	kamera line 1 + AI	line 1 berhenti menilai, sisanya jalan
ripe_line_2	8002	kamera line 2 + AI	sama
ripe_line_3	8003	kamera line 3 + AI	sama
palmgrade_console	8000	layar operator (/console)	operator buta, tapi line tetap menilai dan menyimpan

Line dan konsol sengaja pakai **modul ASGI yang beda**: `main.py` buat line (muat torch, OpenCV, driver kamera) dan `console_main.py` buat konsol (**haram** impor torch/cv2). Alasannya satu: kamera yang rewel jangan sampai menjatuhkan layar operator.

4. Perjalanan satu janjang, dari cahaya sampai angka

Ini inti sistemnya. Kalau cuma satu bagian yang kamu hafal, hafalin ini.



1. **Ambil frame.** `FrameCaptureWorker` minta frame ke kamera dengan irama tetap. Iramanya datang dari kameranya sendiri — lihat §8 soal `.mfs`. Frame masuk antrian yang **buang yang paling lama** kalau penuh, jadi AI selalu dapat gambar terbaru, bukan gambar basi.
2. **Nilai.** `FrameProcessingWorker` jalanin YOLO + ByteTrack. ByteTrack yang bikin satu buah yang kelihatan di 20 frame berturut-turut **dihitung sekali**, bukan 20 kali. Buah dihitung pas dia masuk kotak ROI (area tengah conveyor).
3. **Simpan ke disk dulu, baru kirim.** Ini aturan paling penting di repo ini. Worker deteksi **nggak pernah** ngomong ke jaringan. Dia nulis gambar WebP + JSON ke `artifacts/results/`, terus satu baris ke `outbox.db`. Titik.
4. **Antrian yang ngomong ke jaringan.** `OutboxRetryWorker` ngecek `outbox.db` tiap detik dan ngirim ke konsol. Kalau konsolnya lagi mati, barisnya nunggu. Nggak ada yang hilang.
5. **Konsol nyimpen ke index.** Konsol nulis ke `state/console.db` (SQLite). Layar operator baca dari situ, **nggak pernah** nyisir folder — nyisir folder tiap 2 detik bakal makan I/O yang dipakai buat grading.
6. **Jam-jaman, foto naik ke cloud.** `BatchUploadWorker` ngirim foto ke Cloudflare R2 dan teksnya ke API cloud, sejam sekali. Ini jalur terpisah dari nomor 4 dan boleh telat.

Kenapa ribet begini? Karena listrik mati, internet putus, dan container di-restart itu **kejadian biasa** di pabrik, bukan kasus langka. Disk dulu baru jaringan artinya mati listrik di tengah jalan cuma bikin telat, bukan bikin data hilang.

5. Perjalanan satu truk

Janjang itu butiran kecil. Yang masuk pembukuan adalah **kunjungan truk**:

#	Kejadian	Yang terjadi di sistem
1	Truk naik timbangan bawa muatan	Timbangan → konsol → ERP: tahap <code>gate</code> (berat bruto + jam masuk)
2	Operator pasang truk ke line	Konsol bilang ke line: mulai sekarang janjang dihitung atas nama truk ini
3	Buah dituang, kamera menilai	Janjang numpuk di <code>console.db</code> , ERP belum dikasih tahu apa-apa
4	Operator lepas truk dari line	Konsol → ERP: tahap <code>grading</code> (total, ACC, REJ, persennya)
5	Truk naik timbangan lagi (kosong)	Konsol → ERP: tahap <code>departed</code> (berat tara + jam keluar)
6	ERP menghitung	Neto = bruto – tara, potongan, harga, jadi Purchase Receipt

Satu kunjungan = satu pesan `upsert_visit`, dikirim tiga kali seiring kunjungannya jalan. Tiap kiriman **mengganti** bagian yang dibawanya — makanya bagian yang belum kita punya **tidak dikirim** sama sekali, karena bagian kosong bakal menghapus isi di ERP.

Buah yang ditolak dinaikin lagi ke truk dan ikut pulang. Jadi waktu truk ditimbang keluar, buah tolakan itu ikut nambah berat tara — otomatis nggak ikut dibayar. Rapi banget sebenarnya.

6. Isi tiap folder

Folder utama

Folder	Isinya	Kapan kamu buka
src/palmgrade/	seluruh kode Python	tiap ganti perilaku
tests/	unit, e2e, integration	tiap ganti perilaku (tulis test dulu)
docs/	dokumen dalam, termasuk berkas ini	pas butuh detail
scripts/	skrip bantu (kiosk, seed, build engine, smoke test)	jarang
config/camera/	hikrobot.mfs — setelan kamera, termasuk fps	pas ganti fps atau exposure
sdk/	SDK Hikrobot (MVS), ikut repo biar build Docker nggak butuh internet	hampir nggak pernah
models/release/	berkas model YOLO .pt	pas ganti model
engines/	engine TensorRT hasil build per GPU	nggak pernah manual, make build-engine yang isi
images/	sample_sawit.jpg , gambar contoh buat CAMERA_TYPE=photo	pas tes tanpa kamera
artifacts/	hasil runtime: foto + JSON + outbox.db	pas mau lihat buktinya
state/	console.db , erp_outbox.db , upload_manifest.db	pas mau lihat isi antrean

models/ , engines/ , artifacts/ , state/ **nggak masuk git** — isinya besar, beda tiap mesin, dan sebagian rahasia.

Dalam src/palmgrade/

Kodenya berlapis, dan lapisannya **nggak boleh loncat**: routes → controllers → services → repositories / pipelines / integrations

Folder	Tugasnya	Contoh isi
core/	setelan dan sambungan	config.py (semua env var), dependencies.py , constants.py
routes/	daftar endpoint HTTP doang	console.py , internal.py , health.py
controllers/	terima request, terusin ke service	capture_controller.py
services/	alur bisnis	console_service.py (paling gemuk), erp_queue.py
repositories/	baca-tulis disk & SQLite	console_repository.py , capture_repository.py
pipelines/	inferensi YOLO	model_registry.py , realtime_inspection_pipeline.py
workers/	proses latar yang jalan terus	capture, processing, display, outbox, batch upload, worker ERP
integrations/	dunia luar	camera/ , erp/ , notifications/ , storage/ , upload/ , outbox/ , scheduler/
domain/	aturan murni, not I/O	working_day.py , ffb_source.py , vision_event.py , plate.py
schemas/	bentuk request/response (Pydantic)	internal_schema.py
plc/	Modbus-TCP ke PLC, berdiri sendiri, mati by default	plc/
license/	penjaga langganan (Ed25519), opsional	manager.py , guard.py
static/	console.html — layar operator, satu berkas , tanpa build, tanpa CDN	pas ubah tampilan

Dua folder yang gampang salah paham:

- **domain/** itu tempat aturan yang harus benar walau nggak ada listrik. Isinya fungsi murni: masuk angka, keluar angka. Nggak ada baca disk, nggak ada HTTP. Ini yang paling gampang dites dan paling mahal kalau salah — misalnya `working_day.py` yang nentuin satu shift malam masuk tanggal kerja yang mana.
- **console.html** sengaja satu berkas tanpa framework. Bukan karena malas: layar ini harus hidup saat internet mati, dan halaman yang narik React dari CDN bakal blank persis di saat paling nggak boleh blank.

Berkas di akar repo

Berkas	Gunanya
Makefile	semua perintah lewat sini — <code>make up</code> , <code>make restart</code> , <code>make logs-1</code> , <code>make console</code>
Dockerfile	resep image (torch CPU buat dev, CUDA buat pabrik)
<code>docker-compose.yml</code>	4 service: 3 line + konsol
<code>docker-compose.override.yml</code>	penyimpangan per mesin, dibaca compose otomatis , di-gitignore
<code>.env.example</code>	contoh semua setelan; <code>.env</code> asli nggak masuk git
<code>requirements.txt</code>	dependency Python, dipatok versinya
CLAUDE.md	peta aturan buat AI agent (dan manusia buru-buru)
README.md	cara pasang dan jalanin, panjang

7. Cara menjalankan

Di Mac (buat ngoding konsol)

```
cd autograde
make console          # http://127.0.0.1:8100/console
```

Konsol native, tanpa Docker. Port 8100, bukan 8000, karena 8000 dipakai ERP lokal. `make up` dan `make up-dev` **jangan** dipakai di Mac: butuh SDK kamera dan GPU NVIDIA.

Test:

```
.venv/bin/pytest tests/unit          # jangan tambah -q, pyproject udah masang
.venv/bin/ruff check <daftar di ci.yml>
```

Di PC pabrik (Linux + GPU)

```
make up          # build + bangun engine TensorRT + nyalain 3 line
make ps          # lihat status
make logs-1      # log line 1
make restart     # sesudah ubah kode (kode di-bind-mount, nggak perlu rebuild)
make start       # sesudah ubah .env atau override
```

Aturannya gampang diingat: **ubah kode** → `make restart`. **Ubah setelan** → `make start`. **Ubah dependency atau Dockerfile** → `make up`.

8. Jebakan yang udah makan korban

Tiap baris di sini pernah bikin orang kehilangan waktu berjam-jam.

1. **Fps kamera cuma hidup di satu tempat:** `config/camera/hikrobot.mfs` . Berkas itu dikirim ke kamera tiap nyambung, terus lajunya dibaca balik dari kamera. `CAMERA_FPS` di `.env` itu **cadangan** buat sumber yang nggak bisa lapor (webcam, file video). Dulu keduanya hidup bareng dan yang lebih kecil menang — bikin orang mikir `CAMERA_FPS` rusak berbulan-bulan.
2. **Port 8000 di Mac itu ERP, bukan konsol.** Konsol di Mac selalu 8100. Di PC pabrik, konsol di dalam Docker memang 8000.
3. **BACKEND_URL wajib nunjuk API lokal.** Pernah kejadian ditunjuk ke API cloud dan produksi kebanjiran ~1.098 event tes dalam sehari.
4. **Label produksi itu ACC dan REJ** , bukan `MATANG / MENTAH` . Nilai di luar dua itu ditolak 400 waktu masuk, karena angka ini yang jadi uang.
5. **neto_kg selalu dihitung, nggak pernah dipercaya mentah.** Kalau pengirim ngasih netto yang beda dari bruto – tara lebih dari 1 kg, kiriman ditolak. Dua sumber kebenaran yang diam-diam beda itu cara paling rapi buat salah bayar berbulan-bulan.
6. **Tanggal kerja dihitung pas event masuk, terus disimpan.** Pabrik jalan ~20 jam sehari dan lewat tengah malam, jadi kalau tanggalnya dihitung dari `now()` waktu dibaca, satu shift kepotong jadi dua hari.
7. **Nama image GHCR tetap palmgrade-vision** walau reponya udah ganti nama jadi `autograde` . Ikut ganti = PC pabrik minta nama lama selamanya dan updater-nya jawab "udah paling baru", tanpa error. Dijaga test.
8. **Jangan git pull sesudah ngedit .mfs di PC pabrik** tanpa `git checkout` dulu — berkas itu dilacak git dan editan lokalnya bakal berantem.

9. Aturan kerja

- **Semua perubahan lewat PR.** Branch baru → PR **squash** ke `staging` → PR **merge commit** ke `main` . Jangan commit langsung ke `staging` .
- **Judul dan isi PR wajib bahasa Inggris.** Pesan commit boleh Indonesia.
- **Test dulu, kode belakangan.** Tulis test yang gagal, lihat dia gagal dengan alasan yang benar, baru betulin. Test yang nggak pernah merah itu nggak menjaga apa-apa.
- **Unit test haram narik torch, cv2, atau hardware.** CI jalan di runner kecil tanpa GPU. Butuh yang berat? Taruh di `tests/e2e/` .
- **Komentar dalam bahasa Inggris, singkat, dan jelasin kenapa** — bukan mengulang apa yang udah kelihatan dari kodenya.
- **Jangan pernah tulis "Co-Authored-By: Claude"** atau referensi AI apa pun di commit.

10. Mau baca lebih dalam ke mana

Pertanyaan	Berkas
Aturan padat buat AI agent	CLAUDE.md
Alur dalam, diagram, semua invariant plus alasannya	docs/overview.md
Batas antar lapisan	docs/architecture.md
Daftar endpoint, event, dan env var	docs/backend-overview.md
Pasang dari nol di PC pabrik	docs/SETUP.md
Spesifikasi kamera	docs/camera-spec.md
PLC / ODOT	docs/plc-integration.md
Kontrak ke ERP	../autoerp/docs/autograde-integration.md
Status kerjaan hari ini	../docs/PROGRESS-AUTOGRADE-AUTOERP.md

11. Kamus

Istilah	Artinya
TBS	Tandan Buah Segar — buah sawit yang baru dipanen
janjang / tandan	satu buah sawit utuh, satuan yang dinilai kamera
PKS	Pabrik Kelapa Sawit
ACC / REJ	diterima / ditolak, hasil penilaian AI
tangkai panjang	buah diterima tapi tangkainya kepanjangan — nambah berat, bukan minyak
mentah	buah belum matang, minyaknya sedikit
brondolan	buah lepasan yang rontok dari tandan
line	satu conveyor dengan satu kamera; ada tiga
bruto / tara / neto	berat truk isi / truk kosong / selisihnya = yang diterima pabrik
sortasi	proses penilaian mutu buah
potongan	pengurangan harga karena mutu jelek
Sumber TBS	Internal (kebun sendiri) atau External (beli dari pemasok)
kunjungan	satu siklus truk: masuk, tuang, keluar
outbox	antrean di disk buat hal yang belum sempat terkirim

12. Lima menit pertama kamu

1. Buka `CLAUDE.md` , baca bagian **Critical Rules**. Sepuluh menit, tapi nyelametin berhari-hari.
2. Jalanin `make console` di Mac, buka layarnya, klik-klik.
3. Buka `src/palmgrade/workers/frame_processing_worker.py` — di situ buah berubah jadi angka.
4. Buka `src/palmgrade/domain/working_day.py` — contoh kecil dan bersih dari aturan `domain/` .
5. Jalanin `.venv/bin/pytest tests/unit` , lihat semuanya hijau, baru mulai ngoding.

Kalau ada yang bikin bingung atau kelihatan salah: **itu temuan, bukan "mungkin emang gitu"**. Catat.